

System architecture

How the final frontend, local API, hybrid versioning engine, Merkle search trie, and SQLite object store fit together.

REVON-H

23 AUGUST 2026

Core idea

Revon stores structured states as immutable, content-addressed objects. A commit rewrites only trie paths touched by changed keys, then atomically advances HEAD.

FINAL FRONTEND

React / Vinext workspace | import | history | compare | metrics



DEPENDENCY-FREE API

HTTP + JSON | validation | CORS | optimistic concurrency



REVON-H ENGINE

fixed-depth Merkle hash trie + operation changesets + adaptive diff



DURABLE OBJECT STORE

SQLite transactions | objects by hash | versions | HEAD | commit stats

1. Component boundaries

Each layer has a narrow responsibility. This keeps the research model inspectable while still providing a complete platform.

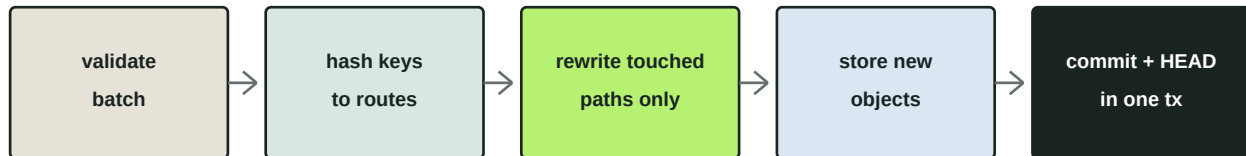
Layer	Responsibility	Key implementation
Frontend	Collect actions and visualize versions, diffs, work, storage, and integrity.	React / Vinext; typed fetch client
HTTP API	Validate requests, map errors, constrain paths, coordinate repository writes.	Python standard library server
Revon-H	Create versions, incremental Merkle updates, changesets, checkouts, adaptive diffs.	VersionedDatabase
Persistence	Durably store immutable objects and mutable references atomically.	SQLiteRevonRepository
Evaluation	Execute identical fixed-seed workloads across baselines and Dolt.	experiments/final_benchmark.py

Trust boundary

The submitted system is intentionally local-first. The backend listens on 127.0.0.1, verifies repository names and stored hashes, and accepts only allowed browser origins. Public hosting of the data service requires authentication, TLS, rate limiting, and deployment-grade coordination; those controls are outside the current research prototype.

2. Incremental commit workflow

Yes: the current model performs incremental hashing. It does not rebuild the complete tree for each normal mutation batch.



Unchanged subtrees retain the same hashes

Cost tracks changed paths, not the size of the complete dataset.

Step	Invariant
1. Validate	Reject conflicting puts/deletes, invalid deletes, or a stale base version before changing state.
2. Route	SHA-256(key) supplies deterministic branch slots for every trie level.
3. Copy paths	Only nodes on routes touched by the batch are reconstructed; siblings keep their hashes.
4. Address	Canonical node, changeset, and commit payloads receive content hashes.
5. Persist	New objects, version mapping, statistics, and HEAD advance in one SQLite transaction.

Failure behavior

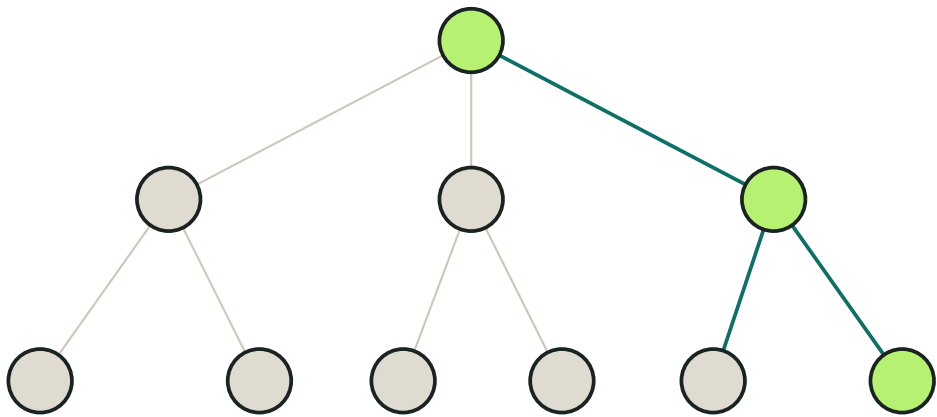
If persistence fails, SQLite rolls back and the in-memory HEAD is restored. Reopening the repository reproduces history, checkout, diffs, and metrics.

3. Fixed-depth Merkle search trie

Default configuration uses branching factor 8 and depth 4. The route consumes three SHA-256 bits per level; leaf buckets retain full keys to resolve route collisions.

ONE KEY CHANGES

acid = rewritten | grey = reused by hash



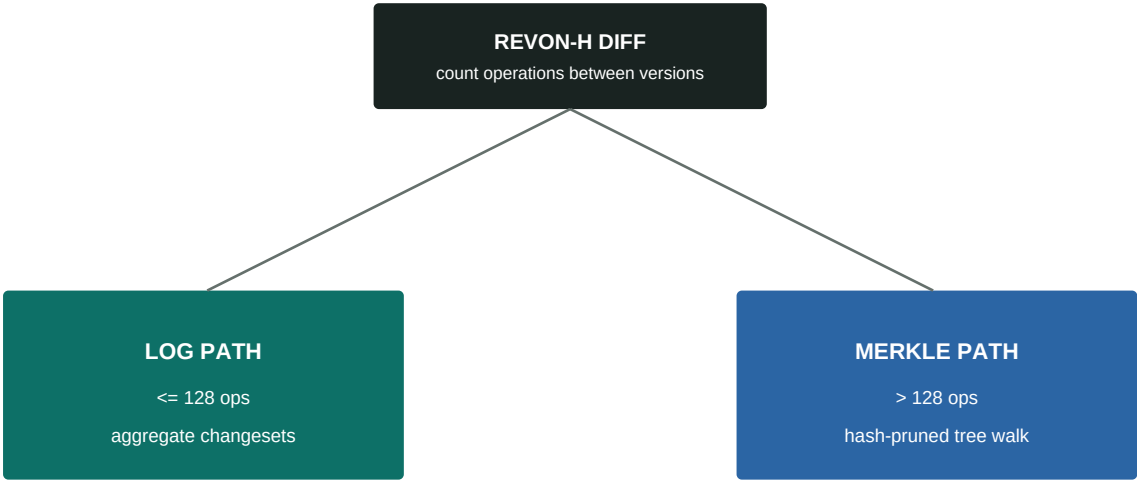
Property	Effect
Deterministic routing	The same key always reaches the same path on every machine.
Canonical encoding	Equivalent structured content produces the same hash.
Structural sharing	Unchanged nodes are referenced by existing hashes instead of copied.
Hash-pruned comparison	Equal subtree hashes prove equality and stop traversal immediately.
Fixed depth	Predictable update path length and a deliberately simple research implementation.

Research trade-off

Compared with Dolt's production Prolly Trees, this trie is simpler and exposes clear path-copying and hash-pruning metrics. Its fixed geometry can create uneven leaf buckets under some key distributions. That limitation belongs in the paper's threats-to-validity section; replacing it now would destroy the controlled identity of Revon-H and exceed the final-submission schedule.

4. Hybrid diff strategy

Revon-H stores both Merkle roots and per-commit operation changesets, then selects the cheaper representation using a calibrated operation threshold.



Both paths return the same structured added / modified / deleted result

Mode	Best case	Reported work unit
Forced Log	Nearby versions with few intervening operations.	log operations examined
Forced Merkle	Large histories or sparse differences across large states.	nodes / leaf entries examined
Revon-H	Selects Log at or below 128 operations; Merkle above it.	selected strategy + its natural work

Correctness rule	Forced Log, forced Merkle, and Revon-H must return the identical structured difference set. The benchmark checks results against a canonical oracle before recording performance.
------------------	---

5. SQLite object model and request flow

SQLite is used as a durable local object database, not as a replacement for the Merkle model. Content addressing remains the versioning layer.

Table	Stores
metadata	branching factor, tree depth, and hybrid threshold
objects	canonical payloads keyed by hash; kinds are node, changeset, or commit
versions	monotonic version number to commit hash
commit_stats	changed keys, new/reused nodes, and bytes written
refs	mutable named reference; currently HEAD

End-to-end request

Browser	API	Engine / store	Response
Commit batch	validate + lock	incremental rewrite + SQLite transaction	new HEAD + commit metrics
Compare vA/vB	parse strategy	Log aggregation or Merkle pruning	changes + work metrics
Checkout vN	validate pagination	resolve commit root + materialize	historical keyed state
Open repo	derive safe path	reopen + verify hashes/references	HEAD + integrity counts

Architecture summary

The frontend demonstrates the product; the API enforces safe workflows; Revon-H supplies the research contribution; SQLite makes the history durable; and the benchmark harness supplies comparable evidence against Snapshot, Log-only, forced-Merkle, and Dolt.